PREDICTIVE ANALYTICS (INT234)

PROJECT REPORT

(Project Semester August-December 2026)

(STEEL PLANT ELECTRICITY USAGE ANALYSIS AND PREDICTION USING MACHINE LEARNING)

Submitted by

Nartu Karthik

Registration No. 12511716

Roll No. RK2P24NKA34

Programme and Section P132-L: B.Tech. (Computer Science and Engineering, Data Science), Section K2P24NK

Course Code INT234

Under the Guidance of

Dr. Mrinalini Rana, U.Id: 22138, Designation

Discipline of CSE

Lovely School of Computer Science and Engineering

Lovely Professional University, Phagwara

CERTIFICATE

This is to certify that Nartu Karthik bearing Registration no. 12511716 has completed INT234 project titled, "STEEL PLANT ELECTRICITY USAGE ANALYSIS AND PREDICTION USING MACHINE LEARNING" under my guidance and supervision. To the best of my knowledge, the present work is the result of his original development, effort and study.

Signature and Name of the Supervisor
Dr. Mrinalini Rana (U.Id: 22138)
Designation of the Supervisor: Designation
School of Computer Science and Engineering
Lovely Professional University
Phagwara, Punjab.

Date: DD/MM/YYYY

DECLARATION

I, Nartu Karthik, student of B.Tech. (Computer Science and Engineering, Data Science) under CSE Discipline at Lovely Professional University, Punjab, hereby declare that all the information furnished in this project report is based on my own intensive work and is genuine.

Date: DD/MM/YYYY Signature

Registration No. 12511716 Nartu Karthik

ACKNOWLEDGEMENT

I would like to thank my faculty guide Dr. Mrinalini Rana for her guidance and support during this project. She explained the course INT234 Predictive Analytics very well and this helped me a lot to complete the work.

I also thank Lovely Professional University and the School of Computer Science and Engineering for giving me this chance to do a practical project. I thank the people who shared the Steel Industry Energy Consumption dataset publicly, and my family and friends for their support.

Nartu Karthik
Registration No. 12511716

TABLE OF CONTENTS

Certificate....2
Declaration....3
Acknowledgement....4
List of Figures....6
List of Tables....7
1. Introduction....8
1.1 About the project....8
1.2 Problem statement....8
1.3 Objectives....8
1.4 Tools and libraries used....8
1.5 Organisation of the report....9
2. Source of Dataset....10
2.1 About the dataset....10
2.2 Columns of the dataset....10
2.3 Some terms used in the dataset....10
2.4 Why I selected this dataset....11
3. EDA Process....12
3.1 Loading the data....12
3.2 Data cleaning....12
3.3 Feature engineering....13
3.4 Understanding the data....14
4. Analysis on Dataset....16
4.1 Analysis 1: Energy usage in the load types....16
4.2 Analysis 2: Weekday and weekend usage....17
4.3 Analysis 3: Busiest hours and months....18
4.4 Analysis 4: Reactive power, power factor and usage....19
4.5 Analysis 5: Predicting the energy usage....21
4.6 Website....24
5. Conclusion....27
5.1 Findings....27
5.2 Recommendations....27
5.3 Limitations....27
6. Future Scope....28
7. References....29
APPENDIX A: LINKEDIN AND GITHUB PROOF....30
APPENDIX B: PYTHON CODE....32

LIST OF FIGURES

Figure 1: Box plots of the main numeric columns to check outliers.....13
Figure 2: Distribution of energy usage and number of readings in each load type.....14
Figure 3: Correlation heatmap of the numeric columns.....15
Figure 4: Energy usage in Light, Medium and Maximum load periods.....17
Figure 5: Weekday and weekend usage, and average usage in each hour.....18
Figure 6: Average energy usage in each month of 2018.....19
Figure 7: Energy usage against lagging reactive power.....20
Figure 8: Comparison of the four regression models.....23
Figure 9: Actual and predicted usage of the best model.....23
Figure 10: Feature importance of the Random Forest model.....23
Figure 11: Home page of the website.....24
Figure 12: Graphs on the website.....25
Figure 13: Predictor on the website.....25
Figure 14: All graphs of the project shown on the website.....26
Figure 15: LinkedIn post of the project with likes.....30
Figure 16: LinkedIn comments on the project post.....30
Figure 17: GitHub repository of the project.....30
Figure 18: GitHub commits of the project.....31
Figure 19: GitHub pull requests of the project.....31
Figure 20: GitHub stars of the project.....31

LIST OF TABLES

Table 1: Tools and libraries used in the project....8
Table 2: Columns of the dataset....10
Table 3: Data cleaning steps....12
Table 4: Outliers found by the IQR rule....13
Table 5: New columns created....13
Table 6: Summary of energy usage (kWh in 15 minutes)....14
Table 7: Energy usage in each load type (kWh in 15 minutes)....16
Table 8: Weekday and weekend usage....17
Table 9: Busiest and quietest times....18
Table 10: Correlation of usage with other columns....20
Table 11: Average usage in each power factor group....20
Table 12: Results of the four models on the test data....22
Table 13: Training on January to September and testing on October to December....22

1. INTRODUCTION

1.1 About the project

In this project I worked on the electricity usage data of a steel plant. A steel plant uses a lot of electricity and the electricity bill is one of its biggest costs. The plant has a meter that notes down the energy used every 15 minutes, along with some other electrical readings. I used these readings to understand when and why the plant uses more electricity. After that I built machine learning models that can predict the energy usage.

I did the whole analysis in Python using a Jupyter Notebook in VS Code. At the end I also made a small website using Flask, HTML and CSS to show the main results and to try the model.

1.2 Problem statement

The problem is to study the 15 minute electricity readings of a steel plant for the year 2018, find the patterns in them, and predict the energy used (in kWh) in every 15 minute period using machine learning. If the usage is understood well, the plant can plan its work better and control its electricity cost.

1.3 Objectives

I took the following five objectives for this project. Each one is solved in Chapter 4.

1. To check whether Light, Medium and Maximum load periods really use different amounts of energy.
2. To check whether the plant uses less energy on weekends than on weekdays.
3. To find the busiest hours of the day and the busiest months of the year.
4. To find how reactive power and power factor are related to the energy usage.
5. To predict the energy usage using four machine learning models and find the best model.

1.4 Tools and libraries used

Table 1: Tools and libraries used in the project

Tool or library	Why I used it
Python 3 and Jupyter Notebook (VS Code)	To write and run the code step by step
pandas and numpy	To read the data, clean it and do calculations
matplotlib and seaborn	To draw the graphs
scipy	To do the ANOVA test and the t-test
scikit-learn	To build the models and calculate MAE, MSE, RMSE and R2
joblib	To save the trained model

Tool or library	Why I used it
Flask, HTML, CSS and JavaScript	To make the website that shows the results

From the syllabus I used data preprocessing (Unit I), regression and the model performance measures (Unit II), the Decision Tree (Unit III) and the Random Forest (Unit VI).

1.5 Organisation of the report

Chapter 2 explains the dataset. Chapter 3 explains how I cleaned and explored the data. Chapter 4 has the analysis of the five objectives and the website. Chapter 5 gives the conclusion, Chapter 6 the future scope and Chapter 7 the references. The social media proof and the Python code are given in the appendices.

2. SOURCE OF DATASET

2.1 About the dataset

The dataset is called Steel Industry Energy Consumption. It is a public dataset and I downloaded it from Kaggle. The same dataset is also available on the UCI Machine Learning Repository and on Mendeley Data (the links are in Chapter 7). It has the electricity readings of a steel plant, taken every 15 minutes during the year 2018. The file name is Steel_industry_data.csv.

The file has 35,040 rows and 11 columns. One year has 365 days and one day has 96 periods of 15 minutes, so 365 x 96 = 35,040 rows. This matches the file, so no day is missing.

2.2 Columns of the dataset

Table 2: Columns of the dataset

Original column	Name I used	Meaning
date	date	Date and time of the reading (dd/mm/yyyy hh:mm)
Usage_kWh	usage_kwh	Energy used by the plant in the 15 minutes (kWh). This is the column I predict
Lagging_Current_Reactive.Power_kVarh	lag_reactive	Lagging reactive power
Leading_Current_Reactive_Power_kVarh	lead_reactive	Leading reactive power
CO2(tCO2)	co2	Carbon dioxide produced, in tonnes
Lagging_Current_Power_Factor	lag_pf	Lagging power factor (0 to 100)
Leading_Current_Power_Factor	lead_pf	Leading power factor (0 to 100)
NSM	nsm	Number of seconds from midnight
WeekStatus	week_status	Weekday or Weekend
Day_of_week	day_of_week	Name of the day
Load_Type	load_type	Light load, Medium load or Maximum load

2.3 Some terms used in the dataset

- kWh is the unit of electrical energy. It is the unit that is used in the electricity bill.
- Reactive power is the part of the power that does not do any useful work but still flows in the wires. Motors and furnaces create it.
- Power factor tells how well the power is being used. A value near 100 is good and a low value is poor.
- Load type tells how heavily the plant is working: Light, Medium or Maximum.

2.4 Why I selected this dataset

- It is real data from a real plant and it is available on Kaggle, UCI and Mendeley.
- It has more than 35,000 rows with both number columns and category columns, so it is good for EDA and for machine learning.
- It has a real problem in the dates (explained in Chapter 3), so I could show the cleaning of inconsistent data.
- The topic is easy to understand: electricity usage is related to money, so the results are meaningful.
- It has no personal data of any person.

3. EDA PROCESS

EDA means exploratory data analysis. Before building any model, I first checked the data, cleaned it and studied it. I did this in the following steps: loading the data, cleaning, feature engineering and then understanding the data with summary numbers and graphs.

3.1 Loading the data

I loaded the file with pandas read_csv. The data has seven number columns (usage, two reactive powers, CO2, two power factors and NSM), three category columns (week status, day of week and load type) and the date column.

3.2 Data cleaning

The CA asks for handling of missing values, duplicates and inconsistent data. I checked all of them with code and noted every step in a table (Table 3).

Table 3: Data cleaning steps

Step	What I did	Items found
Rename columns	Gave short names to all 11 columns	11
Missing values	Checked with isna()	0
Duplicate rows	Checked with duplicated()	0
Wrong dates (inconsistent data)	Moved midnight readings to the correct day and sorted	365
Day and weekend labels	Compared with the real calendar	0
Impossible values	Power factor outside 0-100 or negative energy	0
Possible meter failure	One row with usage 0 and power factor 0, kept it	1
Outliers	Checked with the IQR rule, kept them because they are real values	17474

Missing values and duplicates

There were no missing values and no duplicate rows in the data.

Wrong dates (inconsistent data)

The column NSM shows the number of seconds from midnight. When NSM is 0, the reading is the 12 AM reading that closes the day. But in the file this reading was given the same day instead of the next day. For example the last row of the file is 31/12/2018 00:00, while the first row is 01/01/2018 00:15. Because of this the dates were not in the correct time order. This happened 365 times, once for every day of the year. I moved these readings to the next day and sorted the data by time. After this the data was in correct order and there were 0 gaps, which means every row is exactly 15 minutes after the previous row.

Labels and values

I compared the day names and the weekday or weekend labels with the real calendar and found no mistake. All power factor values were between 0 and 100 and no energy value was negative. There is only one row where both the usage and the power factor are 0. It may be a meter failure for a moment, so I kept it because it is just one row.

Outliers

I checked the outliers using the IQR rule (Table 4 and Figure 1). A value is an outlier if it is more than 1.5 times the IQR below Q1 or above Q3.

Table 4: Outliers found by the IQR rule

Column	Lower limit	Upper limit	Outliers	Percent
usage_kwh	-68.86	123.29	328	0.94%
lag_reactive	-28.21	53.15	1,059	3.02%
lead_reactive	-3.13	5.22	7,759	22.14%
lag_pf	9.77	152.58	1	0.00%
lead_pf	99.25	100.45	8,327	23.76%

The two leading columns show many outliers only because most of their values are exactly 0 or 100. The high usage values are the real busy periods of the plant, and these are the periods I want to predict. So I did not remove any outlier.

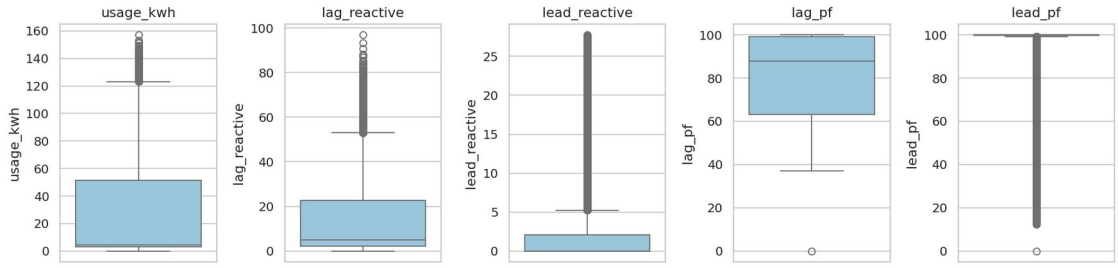


Figure 1: Box plots of the main numeric columns to check outliers

3.3 Feature engineering

I created some new columns from the date and from the electrical readings (Table 5). The date in the file is the end time of the 15 minute period, so I subtracted 15 minutes before taking the hour, month and day.

Table 5: New columns created

New column	How I made it	Why
hour	Hour of the day (0 to 23)	Usage changes with the time of the day
month	Month number	To see the effect of the season
dow_num	Day number (Monday = 0 to Sunday = 6)	To use the day in the model

New column	How I made it	Why
is_weekend	1 for Saturday and Sunday, otherwise 0	Weekend usage is different
shift	Morning (6 to 14), Evening (14 to 22), Night (22 to 6)	To compare the shifts
total_reactive	lag_reactive + lead_reactive	Total reactive power in one number
avg_pf	Average of lag_pf and lead_pf	Power factor in one number

After cleaning and adding the new columns the data has 35,040 rows and 19 columns. I saved it as cleaned_steel_data.csv.

3.4 Understanding the data

Table 6: Summary of energy usage (kWh in 15 minutes)

Measure	Value
Average	27.39
Median	4.57
Standard deviation	33.44
Minimum	0.00
Maximum	157.18
Total in the year	959,637 kWh (about 960 MWh)

The median (4.57) is much smaller than the average (27.39). This means that most of the readings have low usage and a smaller number of readings have very high usage. Figure 2 shows this. The right side of Figure 2 shows the number of readings in each load type: Light load has 18,072, Medium load has 9,696 and Maximum load has 7,272 readings.

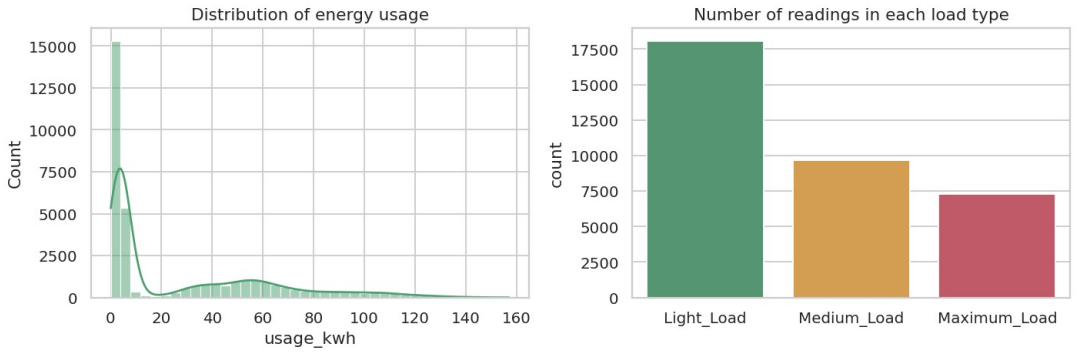


Figure 2: Distribution of energy usage and number of readings in each load type

Figure 3 is the correlation heatmap. Usage has a strong link with the lagging reactive power (r = 0.90) and with CO2 (r = 0.99). CO2 is calculated from the usage, so it is only another form of the same number. If I used CO2 to predict the usage the model would look very good but it would not be a real prediction. So I did not use CO2 in the models.

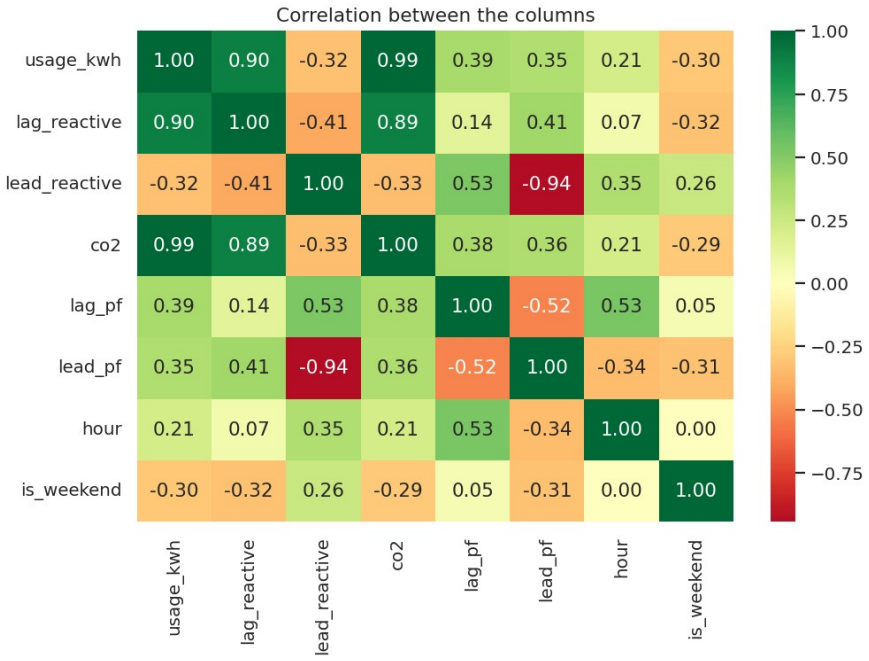


Figure 3: Correlation heatmap of the numeric columns

4. ANALYSIS ON DATASET

Every analysis below is written in five parts: Introduction, General description, Specific requirements (functions and formulas), Analysis results and Visualization.

4.1 Analysis 1: Energy usage in the load types

i. Introduction

Every reading in the dataset is marked as Light, Medium or Maximum load. Before using this column I wanted to check that the three types really use different amounts of energy.

ii. General description

I compared the average usage of the three load types. The hypotheses are: H0, the average usage is the same in all three load types; H1, at least one load type has a different average. There are 18,072 Light, 9,696 Medium and 7,272 Maximum load readings.

iii. Specific requirements, functions and formulas

I used the function scipy.stats.f_oneway for the one-way ANOVA test and the pandas groupby function for the table. The ANOVA formula is:

F = variation between the groups / variation inside the groups

If the p-value is less than 0.05, H0 is rejected.

iv. Analysis results

Table 7: Energy usage in each load type (kWh in 15 minutes)

Load type	Readings	Average	Median	Std	Min	Max
Light	18,072	8.63	3.31	17.91	0.00	140.29
Medium	9,696	38.45	34.44	35.22	2.52	157.18
Maximum	7,272	59.27	56.63	29.75	2.92	151.67

The ANOVA test gave F = 10787.8 and the p-value is less than 0.001. So I reject H0. The average usage is 8.63 kWh in Light load, 38.45 kWh in Medium load and 59.27 kWh in Maximum load. Maximum load uses about 6.9 times the energy of Light load. The ranges overlap a little (the highest Light load reading is 140.29 kWh), so the load type is not decided only by the usage.

v. Visualization

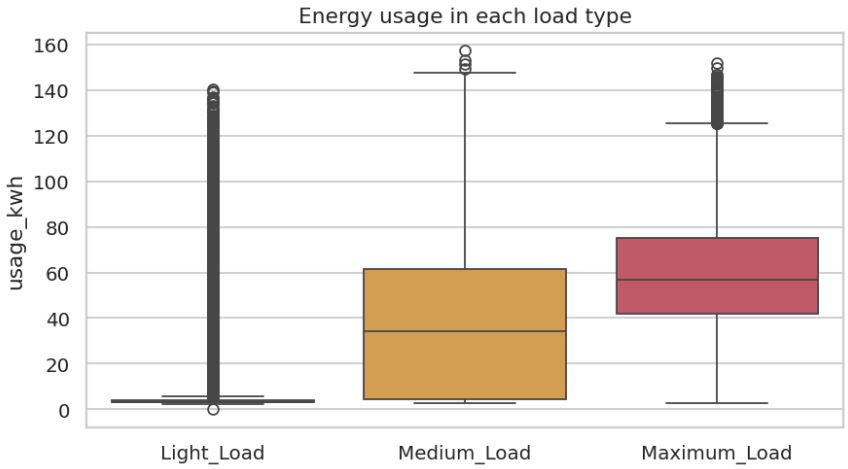


Figure 4: Energy usage in Light, Medium and Maximum load periods

Figure 4 shows a box plot for each load type. The line in the middle of the box is the median. The boxes go higher from Light to Medium to Maximum, which matches the test result.

4.2 Analysis 2: Weekday and weekend usage

i. Introduction

Many factories work less on Sundays and holidays. I wanted to find how much less energy this plant uses at weekends and whether the difference is real or only by chance.

ii. General description

I divided the readings into weekday (25,056 readings) and weekend (9,984 readings) and compared the average usage. H0: the average usage is the same on weekdays and weekends. H1: the averages are different.

iii. Specific requirements, functions and formulas

I used the Welch t-test from scipy.stats.ttest_ind with equal_var=False, because the two groups do not have the same spread. The formula is:

t = (average of group 1 - average of group 2) / standard error

iv. Analysis results

Table 8: Weekday and weekend usage

Measure	Value
Average usage on weekdays	33.62 kWh
Average usage on weekends	11.73 kWh
Weekend usage is lower by	65.1%
t value	72.77
p-value	less than 0.001

The weekend usage is 65.1 percent lower than the weekday usage. The p-value is less than 0.05, so the difference is real and I reject H0.

v. Visualization

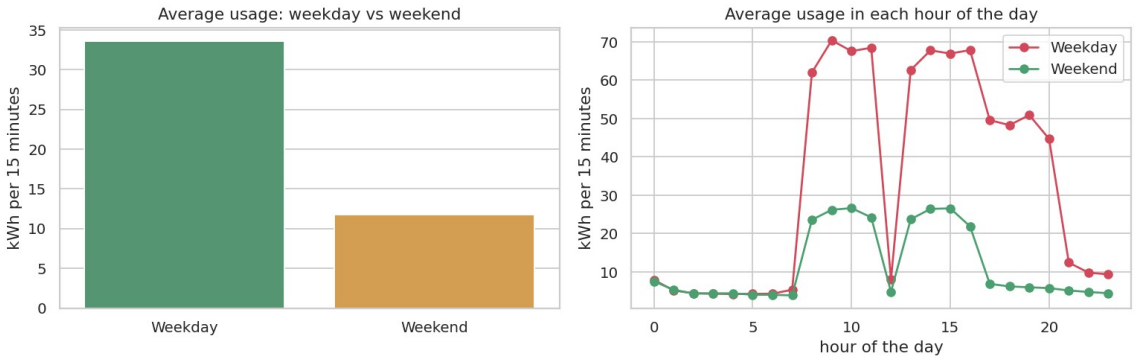


Figure 5: Weekday and weekend usage, and average usage in each hour

The left side of Figure 5 compares the average usage of weekdays and weekends. The right side shows the average usage in every hour of the day. On weekdays the usage is high from the morning to the evening, but on weekends it stays low all day.

4.3 Analysis 3: Busiest hours and months

i. Introduction

If the plant knows its busiest times it can plan its work and its electricity use better. So I looked at the average usage in every hour, every shift and every month.

ii. General description

I grouped the data by hour, by shift and by month and calculated the average usage of each group.

iii. Specific requirements, functions and formulas

I used the pandas groupby function with the mean, and the matplotlib line plot and bar plot. The average of a group is the sum of its values divided by the number of values.

iv. Analysis results

Table 9: Busiest and quietest times

Measure	Result
Busiest hour	9:00 (57.81 kWh in 15 minutes)
Quietest hour	5:00 (4.23 kWh in 15 minutes)
Sudden fall	At 12:00 the usage is only 7.10 kWh, but at 11:00 it is 55.87 kWh
Shifts	Morning 36.07, Evening 40.26, Night 5.84 kWh
Busiest month	Jan (42.42 kWh)

Measure	Result
Quietest month	Dec (19.97 kWh)

The usage is very low at night and rises sharply in the morning. There is a sudden fall at 12:00, which may be the lunch break of the plant. The evening shift has the highest average usage and the night shift has the lowest. Among the months, Jan is the busiest and Dec is the quietest. I have only one year of data, so I cannot say whether this monthly pattern repeats every year.

v. Visualization

The hourly graph is the right side of Figure 5. The monthly graph is Figure 6.

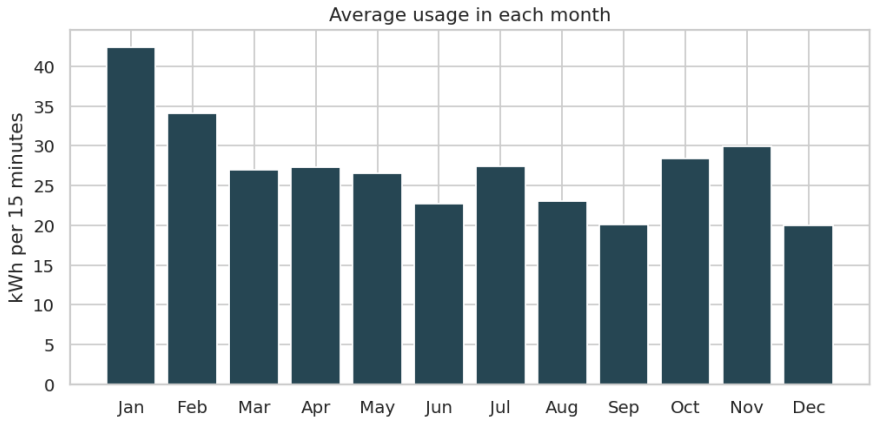


Figure 6: Average energy usage in each month of 2018

4.4 Analysis 4: Reactive power, power factor and usage

i. Introduction

Reactive power and power factor are important measures in an electrical plant. In this analysis I checked how they are related to the energy usage.

ii. General description

I calculated the correlation of the usage with the reactive powers and the power factor. I also divided the lagging power factor into three groups (Poor below 70, Fair 70 to 90 and Good 90 and above) and compared the average usage of each group.

iii. Specific requirements, functions and formulas

I used the pandas corr function, which gives the Pearson correlation r. The value of r is between -1 and +1. A value near +1 means that both columns increase together, and a value near 0 means there is no link.

$$r = \text{covariance}(x, y) / (\text{standard deviation of } x \times \text{standard deviation of } y)$$

iv. Analysis results

Table 10: Correlation of usage with other columns

Column	Correlation (r) with usage
lag_reactive	0.896
lead_reactive	-0.325
lag_pf	0.386
co2	0.988

Table 11: Average usage in each power factor group

Power factor group	Readings	Average usage (kWh)
Fair (70 to 90)	8,479	42.32
Good (90 and above)	15,833	35.50
Poor (below 70)	10,728	3.61

Usage has a strong positive correlation with the lagging reactive power (r = 0.90). It has a medium positive correlation with the lagging power factor (r = 0.39) and a negative correlation with the leading reactive power (r = -0.32). The readings with a poor power factor have a very low average usage (3.61 kWh). So the power factor is poor mostly when the plant is working very little.

v. Visualization

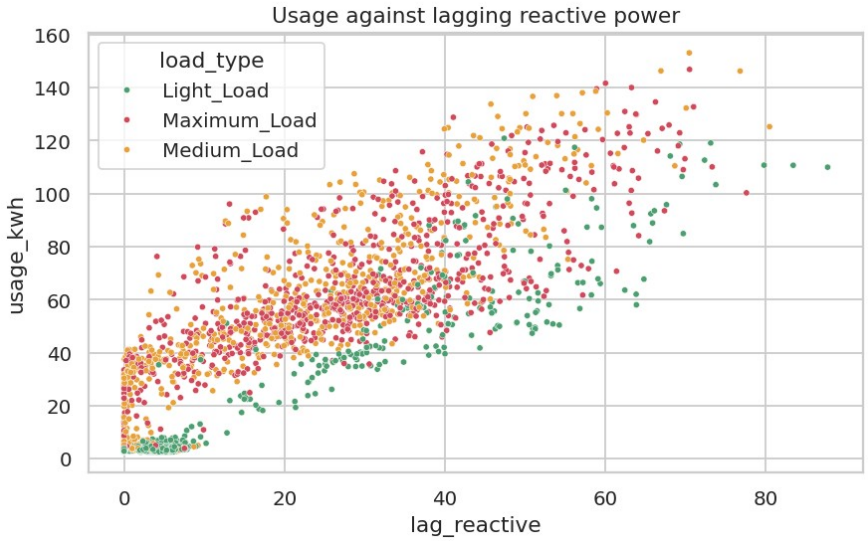


Figure 7: Energy usage against lagging reactive power

Figure 7 is a scatter plot. Each dot is one reading. The dots go upwards from left to right, so when the reactive power is higher the usage is also higher. The Maximum load readings (red) are mostly in the upper right part.

4.5 Analysis 5: Predicting the energy usage

i. Introduction

The last objective is to predict the energy used in a 15 minute period using machine learning. This is a regression problem because the answer (usage_kwh) is a number.

ii. General description

I used ten columns as the input: lag_reactive, lead_reactive, lag_pf, lead_pf, hour, month, dow_num, is_weekend, total_reactive and avg_pf. I did not use CO2 because it is calculated from usage. I did not use the load type because it depends on the load itself. I split the data into 80 percent for training (28,032 rows) and 20 percent for testing (7,008 rows). I used random_state = 42 so the split is the same every time. I trained four models:

- Multiple Linear Regression: it finds the best straight line (a flat surface when there are many inputs) through the data. The formula is y = b0 + b1x1 + b2x2 + ... + bnxn.
- Polynomial Regression (degree 2): it is like linear regression but it also uses the squares and products of the inputs, so it can fit a curve.
- Decision Tree: it asks a series of yes or no questions about the inputs (for example, is lag_reactive less than 12?) and gives the average usage of the group at the end.
- Random Forest: it builds many decision trees (100 in my project), each on a random part of the data, and takes the average of all their answers. This usually gives a better and more stable result than one tree.

iii. Specific requirements, functions and formulas

I used LinearRegression, PolynomialFeatures, DecisionTreeRegressor and RandomForestRegressor from scikit-learn. The inputs of the linear and polynomial models were scaled with StandardScaler. I checked the models with these measures, where y is the real value, y_hat is the predicted value and n is the number of test rows:

MAE = (1/n) x sum of |y - y_hat| *MSE = (1/n) x sum of (y - y_hat)^2*

RMSE = square root of MSE *R2 = 1 - (sum of (y - y_hat)^2) / (sum of (y - average of y)^2)*

MAE, MSE and RMSE show the size of the error, so lower is better. R2 shows how much of the changes in usage the model explains, so a value near 1 is better. I also calculated the R2 on the training data to check overfitting. Overfitting means the model learns the training data too well but does badly on new data.

iv. Analysis results

Table 12: Results of the four models on the test data

Model	MAE	MSE	RMSE	Test R2	Train R2
Random Forest	**0.250**	**0.494**	**0.703**	**0.9996**	**0.9999**
Decision Tree	1.150	5.253	2.292	0.9954	0.9968
Polynomial Regression (degree 2)	2.917	20.594	4.538	0.9819	0.9822
Multiple Linear Regression	7.072	100.896	10.045	0.9112	0.9138

The best model is Random Forest. It has the highest R2 (0.9996) and the lowest errors: MAE = 0.250 kWh and RMSE = 0.703 kWh. The order of the models is Random Forest, then Decision Tree (0.9954), then Polynomial Regression (0.9819) and then Multiple Linear Regression (0.9112). Polynomial regression is better than linear regression, which shows that the link between the inputs and the usage is not a perfect straight line. The train and test R2 of every model are very close, so there is no serious overfitting.

Why the scores are so high

All the models have a high R2 and this is a little unusual. The reason is that in electrical theory the reactive power and the power factor are directly connected with the real power. So the model can almost calculate the usage from them. The feature importance in Figure 10 also shows this: lag_reactive has an importance of 84.4%, followed by avg_pf (9.5%). This means the model is good at estimating the usage when the electrical readings of the same period are known. It is not a forecast of the future, because the reactive power of tomorrow is not known today.

Extra check on new months

To check the model on months it has not seen, I trained it on January to September and tested it on October to December. I removed the month column for this test. The results are in Table 13. The Random Forest still gave an R2 of 0.9996 and the linear regression gave 0.9121, so the results are similar to the earlier ones.

Table 13: Training on January to September and testing on October to December

Model	R2	MAE	RMSE
Multiple Linear Regression	0.9121	7.188	9.473
Random Forest	0.9996	0.248	0.677

v. Visualization

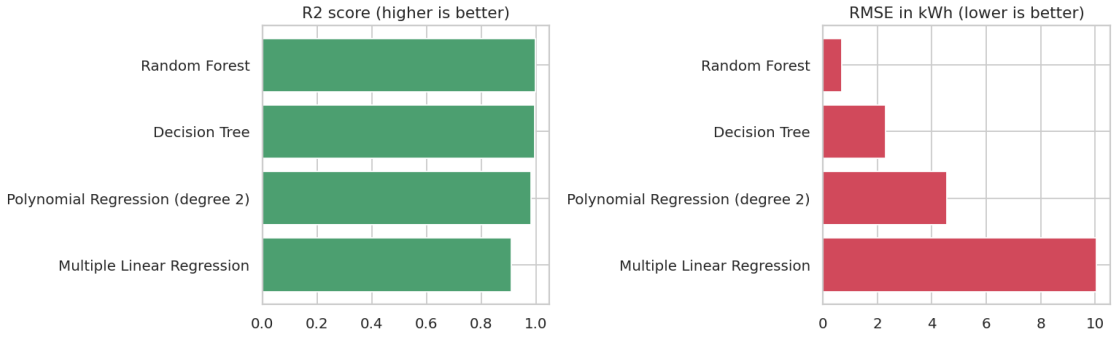


Figure 8: Comparison of the four regression models

Figure 8 compares the four models. The left graph shows R2 (higher is better) and the right graph shows RMSE (lower is better). Random Forest is the best in both.

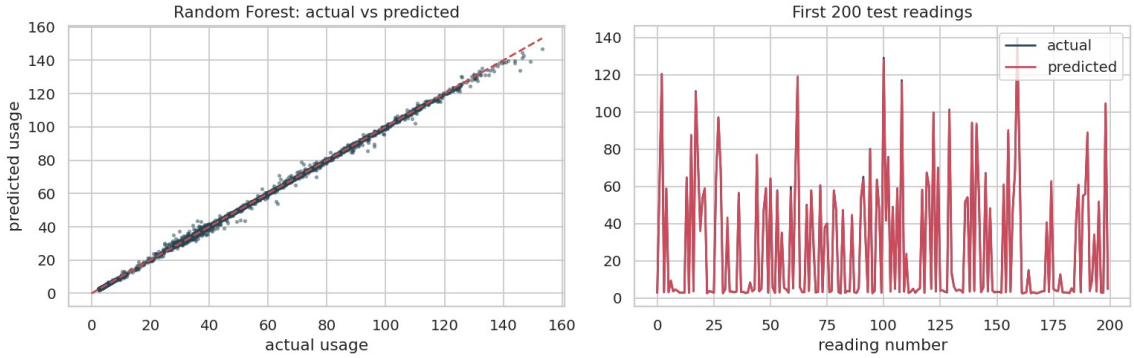


Figure 9: Actual and predicted usage of the best model

Figure 9 compares the real and the predicted usage of the Random Forest on the test data. In the left graph the dots are almost on the red line, which means the predictions are close to the real values. In the right graph the real and the predicted lines almost overlap.

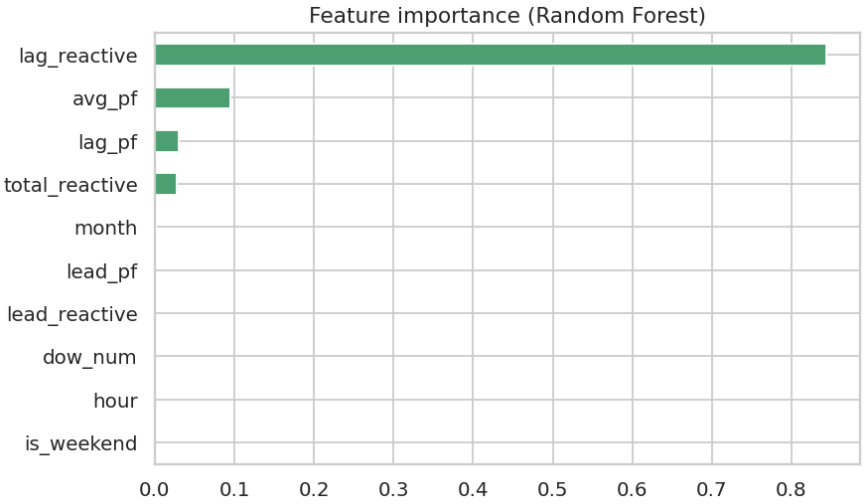


Figure 10: Feature importance of the Random Forest model

Figure 10 shows which inputs the Random Forest depends on most. A longer bar means more importance. The lagging reactive power is the most important input by far.

4.6 Website

I also made a small website to show the results. The backend is written in Flask (app.py). It loads the Random Forest model that was saved by the notebook and gives the data to the page through a few API routes. The frontend is written in HTML, CSS and JavaScript. The page shows the main numbers, the hourly and monthly graphs, the four models, a predictor where we can enter the readings of one period to get the predicted usage, and all ten graphs of the project. The page can be opened by running python app/app.py and opening http://127.0.0.1:5000 in the browser.

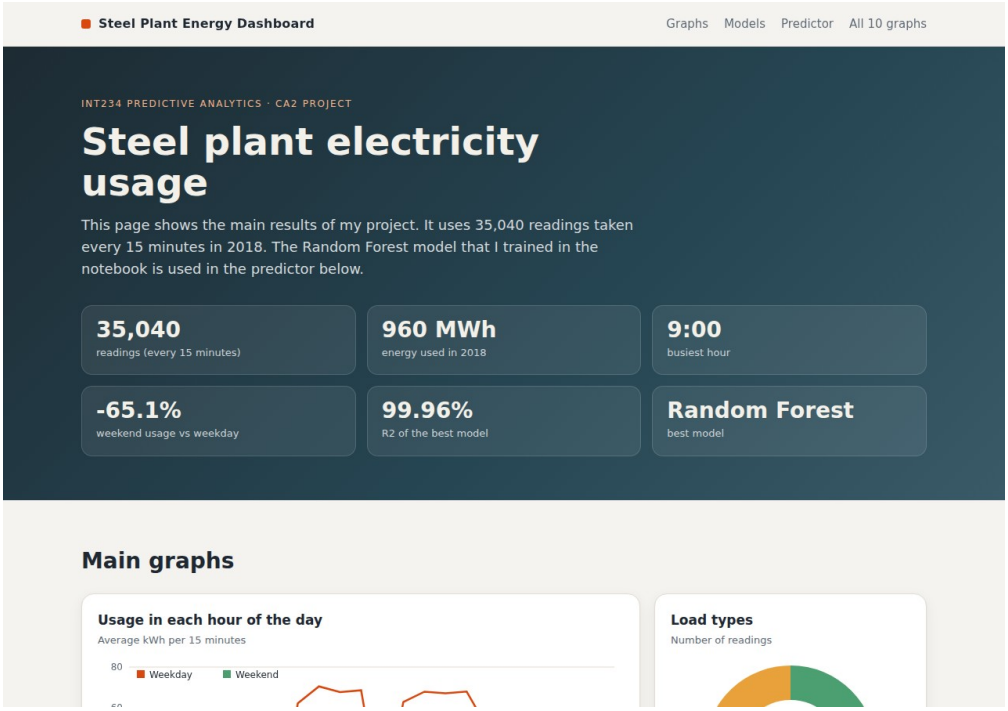


Figure 11: Home page of the website

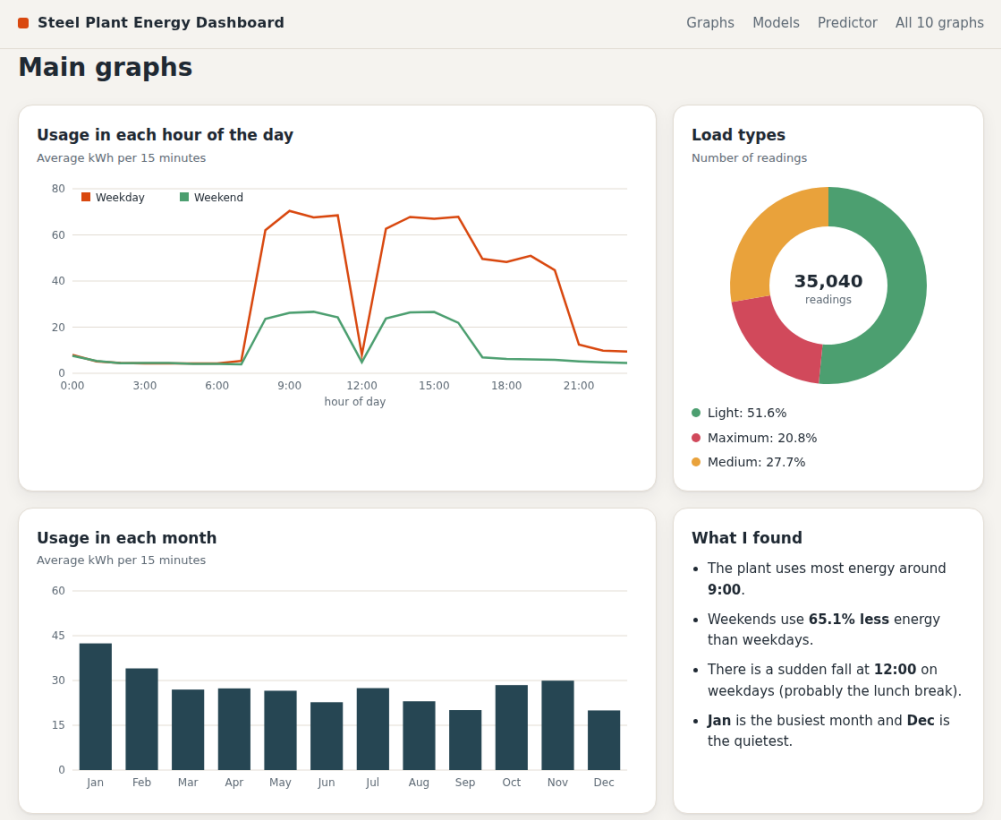


Figure 12: Graphs on the website

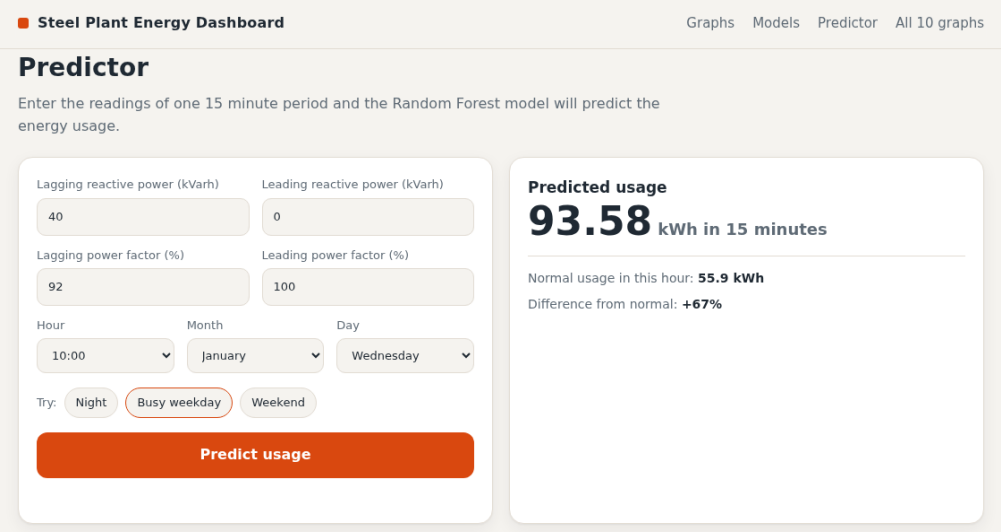


Figure 13: Predictor on the website



Figure 14: All graphs of the project shown on the website

5. CONCLUSION

5.1 Findings

1. The data had no missing values or duplicates, but 365 midnight readings had the wrong date. I corrected them and the data became a complete 15 minute series.
2. The three load types use very different energy. The average is 8.6 kWh in Light load, 38.5 kWh in Medium load and 59.3 kWh in Maximum load (ANOVA, p less than 0.001).
3. The plant uses 65.1 percent less energy at weekends than on weekdays (t-test, p less than 0.001).
4. The busiest hour is 9:00 and there is a sudden fall at 12:00. Jan is the busiest month and Dec is the quietest.
5. Reactive power is strongly linked with usage (r = 0.90). The power factor is poor mostly when the plant is working very little.
6. Of the four models, Random Forest is the best with R2 = 0.9996, MAE = 0.250 kWh and RMSE = 0.703 kWh.

5.2 Recommendations

- The heavy work of the plant can be planned away from the busiest hours of the weekdays, if the production allows it.
- The plant can check what happens at 12:00 when the usage falls suddenly, and whether the machines are switched off or only kept idle.
- The power factor is poor when the plant is nearly idle, so the plant can check whether the power factor can be improved in those periods.
- The model can be used to check the meter readings. If the real usage is very different from the predicted usage, the meter or a machine may have a problem.

5.3 Limitations

The data is of only one plant and one year, so I could not check if the monthly pattern repeats every year. The data does not have the weather, the production or the electricity tariff. The very high R2 is mainly because the reactive power and the power factor are directly connected with the usage, so the model is an estimator and not a real forecast.

6. FUTURE SCOPE

- To use the past usage values (for example the usage of the last hour) so that the next period can be forecast without its electrical readings.
- To add the weather, production and tariff data and calculate the real saving in money.
- To also classify the load type (Light, Medium, Maximum) using classification models such as KNN, SVM and Naive Bayes.
- To use clustering to find the different working modes of the plant.
- To put the website on the internet so that the plant staff can use it, and to add alerts for peak usage.

7. REFERENCES

[1] Steel Industry Energy Consumption dataset, UCI Machine Learning Repository. https://archive.ics.uci.edu/dataset/851

[2] Steel Industry Energy Consumption dataset, Kaggle. https://www.kaggle.com/datasets/csafrit2/steel-industry-energy-consumption

[3] Steel Industry Energy Consumption dataset, Mendeley Data. https://data.mendeley.com/datasets/vts7r3mx79/1

[4] Muller, A. C. and Guido, S. Introduction to Machine Learning with Python (Greyscale Indian Edition). Shroff/O'Reilly.

[5] Geron, A. Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow, 3rd Edition. Shroff/O'Reilly.

[6] scikit-learn documentation. https://scikit-learn.org/stable/

[7] pandas documentation. https://pandas.pydata.org/docs/

[8] SciPy documentation. https://docs.scipy.org/doc/scipy/

[9] Flask documentation. https://flask.palletsprojects.com/

[10] INT234 Predictive Analytics course syllabus, Lovely Professional University, session 2026-27.

APPENDIX A: LINKEDIN AND GITHUB PROOF

LinkedIn profile: https://www.linkedin.com/in/karthik--yadav/

GitHub profile: https://github.com/KarthikYadav-18

GitHub repository of this project: https://github.com/KarthikYadav-18/steel-energy-analytics

The screenshots below show the likes and comments on my LinkedIn posts about this project and my work on GitHub.

INSERT YOUR LINKEDIN POST SCREENSHOT (WITH LIKES) HERE

Figure 15: LinkedIn post of the project with likes

INSERT YOUR LINKEDIN COMMENTS SCREENSHOT HERE

Figure 16: LinkedIn comments on the project post

INSERT YOUR GITHUB REPOSITORY SCREENSHOT HERE

Figure 17: GitHub repository of the project

INSERT YOUR GITHUB COMMITS SCREENSHOT HERE

Figure 18: GitHub commits of the project

INSERT YOUR GITHUB PULL REQUESTS SCREENSHOT HERE

Figure 19: GitHub pull requests of the project

INSERT YOUR GITHUB STARS SCREENSHOT HERE

Figure 20: GitHub stars of the project

APPENDIX B: PYTHON CODE

This is the full code of the Jupyter Notebook (steel_energy_analysis.ipynb) that I used for the analysis.

```
import json
from pathlib import Path
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
import joblib

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, PolynomialFeatures
from sklearn.pipeline import make_pipeline
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# find the project folder (works if you open the notebook from the notebooks
folder or the main folder)
ROOT = Path.cwd() if (Path.cwd() / "data").exists() else Path.cwd().parent
FIG = ROOT / "visualizations"; RES = ROOT / "results"; MOD = ROOT / "models"
for folder in (FIG, RES, MOD):
    folder.mkdir(exist_ok=True)

sns.set_theme(style="whitegrid")
COLORS = {"Light_Load": "#4C9F70", "Medium_Load": "#E9A23B", "Maximum_Load":
"#D1495B"}
R = {}          # all numbers used in the report are collected here
FIGS = []       # list of saved graphs

def save_fig(name, caption):
    plt.tight_layout()
    plt.savefig(FIG / (name + ".png"), dpi=130, bbox_inches="tight")
    FIGS.append({"file": name + ".png", "caption": caption})
    plt.show()
```

```
raw = pd.read_csv(ROOT / "data" / "Steel_industry_data.csv")
print("Rows and columns:", raw.shape)
raw.head()
```

```
raw.info()
```

```
raw.describe().round(2)
```

```
df = raw.copy()
log = []     # small table that records every cleaning step

# 3.1 rename the columns so they are short and easy to type
df.columns = ["date", "usage_kwh", "lag_reactive", "lead_reactive", "co2",
"lag_pf", "lead_pf",
```

```
              "nsm", "week_status", "day_of_week", "load_type"]
log.append(["Rename columns", "Gave short names to all 11 columns", 11])

# 3.2 missing values
missing = int(df.isna().sum().sum())
print("Missing values:", missing)
log.append(["Missing values", "Checked with isna()", missing])

# 3.3 duplicate rows
dups = int(df.duplicated().sum())
print("Duplicate rows:", dups)
log.append(["Duplicate rows", "Checked with duplicated()", dups])
```

```
# 3.4 the date column is text, so I convert it to a real date and time
df["timestamp"] = pd.to_datetime(df["date"], format="%d/%m/%Y %H:%M")
print("Is the data in time order?", df["timestamp"].is_monotonic_increasing)

# nsm means 'number of seconds from midnight'. When nsm is 0 the reading is
the 12 AM reading,
# which closes the day. But the file gives it the SAME day. Example: the last
row says 31/12/2018 00:00.
midnight = df["nsm"] == 0
print("Midnight rows with the wrong day:", int(midnight.sum()))
print("Before fixing, last 3 rows:")
df[["date", "nsm"]].tail(3)
```

```
# Fix: move every midnight reading to the next day and sort by time
df.loc[midnight, "timestamp"] = df.loc[midnight, "timestamp"] +
pd.Timedelta(days=1)
df = df.sort_values("timestamp").reset_index(drop=True)

gaps = int((df["timestamp"].diff().dropna() !=
pd.Timedelta(minutes=15)).sum())
print("In time order now:", df["timestamp"].is_monotonic_increasing)
print("Gaps in the 15 minute series:", gaps)
print("First reading:", df["timestamp"].min(), "| Last reading:",
df["timestamp"].max())
log.append(["Wrong dates (inconsistent data)", "Moved midnight readings to
the correct day and sorted", int(midnight.sum())])
R["midnight_fixed"] = int(midnight.sum()); R["gaps_after_fix"] = gaps
```

```
# 3.5 check that the labels and values make sense
reading_time = df["timestamp"] - pd.Timedelta(minutes=15)       # the time at
which the reading period started
day_mismatch = int((reading_time.dt.day_name() != df["day_of_week"]).sum())
weekend_mismatch = int(((reading_time.dt.dayofweek >= 5) !=
(df["week_status"] == "Weekend")).sum())
print("Day name wrong:", day_mismatch, "| Weekend label wrong:",
weekend_mismatch)
log.append(["Day and weekend labels", "Compared with the real calendar",
day_mismatch + weekend_mismatch])

bad_pf = int(((df[["lag_pf", "lead_pf"]] < 0) | (df[["lag_pf", "lead_pf"]] >
100)).sum().sum())
negative = int((df[["usage_kwh", "lag_reactive", "lead_reactive", "co2"]] <
0).sum().sum())
```

```
print("Power factor outside 0 to 100:", bad_pf, "| Negative energy values:",
negative)
log.append(["Impossible values", "Power factor outside 0-100 or negative
energy", bad_pf + negative])

zero_row = df[(df["usage_kwh"] == 0) & (df["lag_pf"] == 0)]
print("Rows with usage 0 and power factor 0:", len(zero_row))
log.append(["Possible meter failure", "One row with usage 0 and power factor
0, kept it", len(zero_row)])
```

```
# 3.6 outliers using the IQR rule
rows = []
for col in ["usage_kwh", "lag_reactive", "lead_reactive", "lag_pf",
"lead_pf"]:
    q1, q3 = df[col].quantile([0.25, 0.75])
    iqr = q3 - q1
    low, high = q1 - 1.5 * iqr, q3 + 1.5 * iqr
    count = int(((df[col] < low) | (df[col] > high)).sum())
    rows.append([col, round(low, 2), round(high, 2), count, round(100 * count
/ len(df), 2)])
outliers = pd.DataFrame(rows, columns=["column", "lower_limit",
"upper_limit", "outliers", "percent"])
R["outliers"] = outliers.to_dict("records")
log.append(["Outliers", "Checked with the IQR rule, kept them because they
are real values", int(outliers["outliers"].sum())])
outliers
```

```
# Graph 1: box plots to see the outliers
fig, axes = plt.subplots(1, 5, figsize=(14, 3.5))
for ax, col in zip(axes, ["usage_kwh", "lag_reactive", "lead_reactive",
"lag_pf", "lead_pf"]):
    sns.boxplot(y=df[col], ax=ax, color="#8ecae6")
    ax.set_title(col)
save_fig("fig01_boxplots", "Box plots of the main numeric columns to check
outliers")
```

```
cleaning_log = pd.DataFrame(log, columns=["step", "what_I_did",
"items_affected"])
R["cleaning_log"] = cleaning_log.to_dict("records")
cleaning_log
```

```
reading_time = df["timestamp"] - pd.Timedelta(minutes=15)
df["hour"] = reading_time.dt.hour                       # hour of the day (0
to 23)
df["month"] = reading_time.dt.month                     # month number
df["dow_num"] = reading_time.dt.dayofweek               # Monday = 0 ...
Sunday = 6
df["is_weekend"] = (df["dow_num"] >= 5).astype(int)     # 1 for Saturday and
Sunday
df["shift"] = np.select([df["hour"].between(6, 13), df["hour"].between(14,
21)], ["Morning", "Evening"], default="Night")
df["total_reactive"] = df["lag_reactive"] + df["lead_reactive"]
df["avg_pf"] = (df["lag_pf"] + df["lead_pf"]) / 2

df.to_csv(ROOT / "data" / "cleaned_steel_data.csv", index=False)
R["clean_rows"], R["clean_cols"] = int(df.shape[0]), int(df.shape[1])
```

```
df[["timestamp", "usage_kwh", "hour", "month", "is_weekend", "shift",
"total_reactive", "avg_pf"]].head()
```

```
R["usage_stats"] = {"mean": float(df.usage_kwh.mean()), "median":
float(df.usage_kwh.median()), "std": float(df.usage_kwh.std()),
                    "min": float(df.usage_kwh.min()), "max":
float(df.usage_kwh.max()), "total": float(df.usage_kwh.sum())}
R["load_counts"] = df["load_type"].value_counts().to_dict()
print(R["usage_stats"])
print(R["load_counts"])
```

```
# Graph 2: how usage is spread and how many readings each load type has
fig, axes = plt.subplots(1, 2, figsize=(11, 3.8))
sns.histplot(df["usage_kwh"], bins=40, kde=True, color="#4C9F70", ax=axes[0])
axes[0].set_title("Distribution of energy usage");
axes[0].set_xlabel("usage_kwh")
sns.countplot(data=df, x="load_type", order=["Light_Load", "Medium_Load",
"Maximum_Load"], palette=COLORS, ax=axes[1])
axes[1].set_title("Number of readings in each load type");
axes[1].set_xlabel("")
save_fig("fig02_distribution", "Distribution of energy usage and number of
readings in each load type")
```

```
# Graph 3: correlation heatmap
cols = ["usage_kwh", "lag_reactive", "lead_reactive", "co2", "lag_pf",
"lead_pf", "hour", "is_weekend"]
corr = df[cols].corr()
plt.figure(figsize=(8, 6))
sns.heatmap(corr, annot=True, fmt=".2f", cmap="RdYlGn", center=0)
plt.title("Correlation between the columns")
save_fig("fig03_correlation", "Correlation heatmap of the numeric columns")
R["corr_with_usage"] = corr["usage_kwh"].round(3).to_dict()
```

```
order = ["Light_Load", "Medium_Load", "Maximum_Load"]
groups = [df.loc[df.load_type == k, "usage_kwh"] for k in order]
f_value, p_value = stats.f_oneway(*groups)
summary = df.groupby("load_type")["usage_kwh"].agg(["count", "mean",
"median", "std", "min", "max"]).loc[order].round(2)
print("ANOVA F =", round(f_value, 1), "| p-value =", p_value)
R["o1"] = {"F": float(f_value), "p": float(p_value), "table":
summary.reset_index().to_dict("records")}
summary
```

```
# Graph 4: usage for each load type
plt.figure(figsize=(7, 4))
sns.boxplot(data=df, x="load_type", y="usage_kwh", order=order,
palette=COLORS)
plt.title("Energy usage in each load type"); plt.xlabel("")
save_fig("fig04_load_type_usage", "Energy usage in Light, Medium and Maximum
load periods")
```

```
weekday = df.loc[df.week_status == "Weekday", "usage_kwh"]
weekend = df.loc[df.week_status == "Weekend", "usage_kwh"]
t_value, p_value = stats.ttest_ind(weekday, weekend, equal_var=False)
drop = 100 * (weekday.mean() - weekend.mean()) / weekday.mean()
```

```
print("Weekday average:", round(weekday.mean(), 2), "| Weekend average:",
round(weekend.mean(), 2))
print("Weekend is", round(drop, 1), "% lower | t =", round(t_value, 2), "| p-
value =", p_value)
R["o2"] = {"weekday_mean": float(weekday.mean()), "weekend_mean":
float(weekend.mean()), "drop_pct": float(drop), "t": float(t_value), "p":
float(p_value),
          "weekday_count": int(len(weekday)), "weekend_count":
int(len(weekend))}
```

```
hourly = df.groupby(["hour", "week_status"])["usage_kwh"].mean().unstack()
hourly_all = df.groupby("hour")["usage_kwh"].mean()
monthly = df.groupby("month")["usage_kwh"].mean()
names = ["Jan", "Feb", "Mar", "Apr", "May", "Jun", "Jul", "Aug", "Sep",
"Oct", "Nov", "Dec"]

# Graph 5: weekday vs weekend (bar) and average usage in every hour (line)
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
sns.barplot(x=["Weekday", "Weekend"], y=[weekday.mean(), weekend.mean()],
palette=["#4C9F70", "#E9A23B"], ax=axes[0])
axes[0].set_title("Average usage: weekday vs weekend");
axes[0].set_ylabel("kWh per 15 minutes")
axes[1].plot(hourly.index, hourly["Weekday"], marker="o", label="Weekday",
color="#D1495B")
axes[1].plot(hourly.index, hourly["Weekend"], marker="o", label="Weekend",
color="#4C9F70")
axes[1].set_title("Average usage in each hour of the day");
axes[1].set_xlabel("hour of the day"); axes[1].set_ylabel("kWh per 15
minutes"); axes[1].legend()
save_fig("fig05_weekday_hour", "Weekday and weekend usage, and average usage
in each hour")
```

```
# Graph 6: average usage in each month
plt.figure(figsize=(8, 4))
plt.bar(names, monthly.values, color="#264653")
plt.title("Average usage in each month"); plt.ylabel("kWh per 15 minutes")
save_fig("fig06_monthly", "Average energy usage in each month of 2018")

R["o3"] = {"peak_hour": int(hourly_all.idxmax()), "peak_hour_value":
float(hourly_all.max()),
          "low_hour": int(hourly_all.idxmin()), "low_hour_value":
float(hourly_all.min()),
          "noon_value": float(hourly_all[12]), "eleven_value":
float(hourly_all[11]),
          "peak_month": names[int(monthly.idxmax()) - 1],
"peak_month_value": float(monthly.max()),
          "low_month": names[int(monthly.idxmin()) - 1], "low_month_value":
float(monthly.min()),
          "shift": df.groupby("shift")
["usage_kwh"].mean().round(2).to_dict(),
          "hourly": {int(k): round(float(v), 2) for k, v in
hourly_all.items()},
          "monthly": {names[int(k) - 1]: round(float(v), 2) for k, v in
monthly.items()}}
print("Busiest hour:", R["o3"]["peak_hour"], "| Quietest hour:", R["o3"]
["low_hour"])
```

```
print("Busiest month:", R["o3"]["peak_month"], "| Quietest month:", R["o3"]
["low_month"])
print(R["o3"]["shift"])
```

```
r_lag = df["usage_kwh"].corr(df["lag_reactive"])
r_lead = df["usage_kwh"].corr(df["lead_reactive"])
r_pf = df["usage_kwh"].corr(df["lag_pf"])
r_co2 = df["usage_kwh"].corr(df["co2"])
print("usage vs lag_reactive:", round(r_lag, 3))
print("usage vs lead_reactive:", round(r_lead, 3))
print("usage vs lag_pf:", round(r_pf, 3))
print("usage vs co2:", round(r_co2, 3))
df["pf_group"] = np.select([df["lag_pf"] < 70, df["lag_pf"] < 90], ["Poor
(below 70)", "Fair (70 to 90)"], default="Good (90 and above)")
pf_table = df.groupby("pf_group")["usage_kwh"].agg(["count",
"mean"]).round(2)
R["o4"] = {"r_lag": float(r_lag), "r_lead": float(r_lead), "r_pf":
float(r_pf), "r_co2": float(r_co2), "pf_table":
pf_table.reset_index().to_dict("records")}
pf_table
```

```
# Graph 7: usage against lagging reactive power
sample = df.sample(4000, random_state=42)
plt.figure(figsize=(7, 4.5))
sns.scatterplot(data=sample, x="lag_reactive", y="usage_kwh",
hue="load_type", palette=COLORS, s=12)
plt.title("Usage against lagging reactive power")
save_fig("fig07_scatter", "Energy usage against lagging reactive power")
```

```
features = ["lag_reactive", "lead_reactive", "lag_pf", "lead_pf", "hour",
"month", "dow_num", "is_weekend", "total_reactive", "avg_pf"]
X = df[features]
y = df["usage_kwh"]

# 80 percent of the rows for training and 20 percent for testing
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
print("Training rows:", len(X_train), "| Testing rows:", len(X_test))
R["split"] = {"train": int(len(X_train)), "test": int(len(X_test)),
"features": features}
```

```
models = {
    "Multiple Linear Regression": make_pipeline(StandardScaler(),
LinearRegression()),
    "Polynomial Regression (degree 2)": make_pipeline(StandardScaler(),
PolynomialFeatures(2, include_bias=False), LinearRegression()),
    "Decision Tree": DecisionTreeRegressor(max_depth=10, min_samples_leaf=10,
random_state=42),
    "Random Forest": RandomForestRegressor(n_estimators=100,
min_samples_leaf=2, n_jobs=-1, random_state=42),
}

results = []
predictions = {}
for name, model in models.items():
```

```
    model.fit(X_train, y_train)                    # learn from the training
data
    pred = model.predict(X_test)                   # predict the test data
    predictions[name] = pred
    mse = mean_squared_error(y_test, pred)
    results.append({"Model": name,
                    "MAE": mean_absolute_error(y_test, pred),
                    "MSE": mse,
                    "RMSE": np.sqrt(mse),
                    "Test R2": r2_score(y_test, pred),
                    "Train R2": r2_score(y_train, model.predict(X_train))})

results = pd.DataFrame(results).sort_values("Test R2",
ascending=False).reset_index(drop=True)
best_name = results.loc[0, "Model"]
print("Best model:", best_name)
R["o5"] = {"table": results.round(4).to_dict("records"), "best": best_name}
results.round(4)
```

```
# Graph 8: compare the 4 models
fig, axes = plt.subplots(1, 2, figsize=(12, 3.8))
order_models = results.sort_values("Test R2")
axes[0].barh(order_models["Model"], order_models["Test R2"],
color="#4C9F70"); axes[0].set_title("R2 score (higher is better)");
axes[0].set_xlim(0, 1.05)
axes[1].barh(order_models["Model"], order_models["RMSE"], color="#D1495B");
axes[1].set_title("RMSE in kWh (lower is better)")
save_fig("fig08_model_comparison", "Comparison of the four regression
models")
```

```
# Graph 9: actual and predicted values of the best model
best_pred = predictions[best_name]
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
axes[0].scatter(y_test, best_pred, s=4, alpha=0.4, color="#264653")
limit = [0, max(y_test.max(), best_pred.max())]
axes[0].plot(limit, limit, "r--"); axes[0].set_xlabel("actual usage");
axes[0].set_ylabel("predicted usage"); axes[0].set_title(best_name + ":
actual vs predicted")
axes[1].plot(y_test.values[:200], label="actual", color="#264653")
axes[1].plot(best_pred[:200], label="predicted", color="#D1495B")
axes[1].set_title("First 200 test readings"); axes[1].set_xlabel("reading
number"); axes[1].legend()
save_fig("fig09_actual_vs_predicted", "Actual and predicted usage of the best
model")
```

```
# Graph 10: which inputs are most important for the Random Forest
rf = models["Random Forest"]
importance = pd.Series(rf.feature_importances_, index=features).sort_values()
plt.figure(figsize=(7, 4.2))
importance.plot(kind="barh", color="#4C9F70")
plt.title("Feature importance (Random Forest)")
save_fig("fig10_feature_importance", "Feature importance of the Random Forest
model")
R["o5"]["importance"] =
importance.sort_values(ascending=False).round(4).to_dict()
importance.sort_values(ascending=False).round(4)
```

```
train_part = df["month"] <= 9
test_part = ~train_part
X2 = X.drop(columns=["month"])
m1 = make_pipeline(StandardScaler(), LinearRegression()).fit(X2[train_part],
y[train_part])
m2 = RandomForestRegressor(n_estimators=100, min_samples_leaf=2, n_jobs=-1,
random_state=42).fit(X2[train_part], y[train_part])
check = []
for name, m in [("Multiple Linear Regression", m1), ("Random Forest", m2)]:
    p = m.predict(X2[test_part])
    check.append({"Model": name, "R2": r2_score(y[test_part], p), "MAE":
mean_absolute_error(y[test_part], p), "RMSE":
np.sqrt(mean_squared_error(y[test_part], p))})
check = pd.DataFrame(check).round(4)
R["o5"]["new_months"] = check.to_dict("records")
check
```

```
# train the final Random Forest on all rows for the website
final_model = RandomForestRegressor(n_estimators=100, min_samples_leaf=4,
n_jobs=-1, random_state=42).fit(X, y)
joblib.dump({"model": final_model, "features": features}, MOD /
"usage_model.joblib", compress=3)

R["figures"] = FIGS
R["dataset"] = {"rows": int(len(df)), "start": str(df["timestamp"].min()),
"end": str(df["timestamp"].max())}
R["raw_rows"], R["raw_cols"] = int(raw.shape[0]), int(raw.shape[1])
(RES / "results.json").write_text(json.dumps(R, indent=2, default=float))
results.round(4).to_csv(RES / "model_results.csv", index=False)
cleaning_log.to_csv(RES / "cleaning_log.csv", index=False)
print("Saved the model, results.json and", len(FIGS), "graphs")
```